

تقرير إنجاز المرحلة صفر — (Foundation) التأسيس

تقرير إنجاز المرحلة

اسم المرحلة ورقمها

Sakanify Backend (V Div): مشروع (Foundation) التأسيس — (Phase 0) المرحلة صفر

الهدف من المرحلة

لحد Phase 1 من) اللي كل المراحل الجاية (Architectural Foundation) بناء الأساس المعماري بس الهيكل العامة — features أو business logic هتتعتمد عليه. المرحلة دي مفياش أي (Phase 8 زي: (shared infrastructure) والبنية التحتية المشتركة، (configuration) الإعدادات، (scaffolding) constants، ال error handling، ال API الاتصال بقاعدة البيانات، شكل الاستجابة الموحد لل jobs الفاضي لحد ما المراحل الجاية تسجل فيه (job scheduler) المشتركة، ومحرك الجدولة

الملفات والمجلدات اللي اتعملت فعلياً

“Init: modular (commit باسم من) جاهز من قبل (folders) المشروع كان فيه هيكل مجلدات لكن كل الملفات كانت فاضية (0 سطر). المرحلة دي ملأت الملفات، (monolith folder structure) package.json و .gitignore و .env. دي فعلياً بالكود، وعدّلت كمان على

sakanify-backend/	
├── .env	(جديد - فيه بيانات الاتصال)
(git الحقيقية، غير متتبع في	
├── .env.example	(اتملى - قالب لكل متغيرات)
(البيئة المطلوبة	
├── .gitignore	(atlas- اتعدّل - إضافة)
credentials.env)	
├── package.json	@aws- اتعدّل - إضافة مكتبة)
sdk/client-s3)	
├── src/	
│ ├── config/	
│ │ ├── env.config.js	(جديد)
│ │ ├── database.config.js	(جديد)
│ │ ├── storage.config.js	(جديد)
│ │ └── constants.config.js	(جديد)
│ ├── middleware/	
│ │ ├── error-handler.middleware.js	(جديد)
│ │ ├── request-logger.middleware.js	(جديد)
│ │ └── rate-limiter.middleware.js	(جديد)
│ └── shared/	

utils/	
date.util.js	(جديد)
response.util.js	(جديد)
file-upload.util.js	(جديد)
jobs/	
scheduler.core.js	(جديد)
app.entry.js	(جديد)
server.entry.js	(جديد)
Docs/reports/	
phase-0-foundation-report.md	(هذا التقرير)
phase-0-foundation-report.pdf	

ملفات `src/modules/*` و `src/middleware/auth.middleware.js` ملحوظة: موجودة في المشروع من الهيكلة الأولية لكن لم `src/config/queue.config.js` وليست جزءًا (وما بعدها Phase 1) **تلمس في هذه المرحلة**، لأنها تخص مراحل لاحقة Phase 0. من نطاق

شرح تفصيلي لكل خطوة اتبعتها

1. `constants.config.js` — المشتركة التي باقي المديولز هتستوردها: الأدوار `enums` فيه كل الـ (ROLES: student, owner, super-admin)، حالة السرير (BED_STATUS: available, pending, occupied, maintenance)، حالة الطلب (PAYMENT_STATUS: pending, paid, confirmed, overdue)، القيم دي مبنية على القواعد (REQUEST_STATUS: pending, approved, rejected, expired, cancelled). (خاصة قفل السرير الذري وأن الدفع كاش فقط) `overview.md` الأساسية المذكورة في 00
2. `response.util.js` — (success) سواء نجاح، API بيوفر شكل استجابة واحد وموحد لكل الـ { success, message, data } أو { success, message, errors } بالشكل، (error) أو فشل `res.json()` في أي موديول لاحقًا هستخدم الدالتين دول بدل ما ينادي controller أي مباشرة.
3. `date.util.js` — (addDays, addHours, isPast, diffInDays... إلخ) دوال بسيطة لحساب التواريخ وتجديد الاشتراكات (Phase 4) هستخدم لاحقًا في انتهاء صلاحية الطلبات (إلخ) لأن الاحتياج بسيط في (dayjs أو moment زي) اتعملت من غير أي مكتبة خارجية. (Phase 6) المرحلة دي.
4. `file-upload.util.js` — لاستقبال الملفات (صورة البطاقة، صورة multer إعداد مشترك لـ `memoryStorage`) الملفات بتتخزن في الذاكرة. Phase 2 الطالب) هتستخدم فعليًا في (فقط jpeg/png/webp) بعدين، مع فلتر نوع الملف S3 storage الدسك، عشان تتبعت مباشرة لـ 5 ميغا.
5. `error-handler.middleware.js` — من أي error المركزي اللي بيمسك أي middleware الـ `response.util` موديول ويرجعه بشكل موحد عبر Mongoose يتعامل مع: أخطاء التحقق من `validationError`، (duplicate key error 11000) تكرار المفتاح الفريد، (CastError) الغلط Object Id أخطاء الـ (JWT أخطاء الـ) error 11000، (throw new AppError('Bed already reserved', 409)) واضح HTTP جاهز لأي موديول يرميه بكوند حالة `AppError`.

6. request-logger.middleware.js — لوجينج تشغيلي بسيط (method, path, status code, زمن، الذي هيتعمل في موديول (audit log) مش سجل التدقيق التفصيلي، request لكل الاستجابة منفصل بعدين.

7. rate-limiter.middleware.js — express- حماية أساسية من إساءة الاستخدام باستخدام rate-limit، قابل للتعديل عبر متغيرات البيئة، IP، بحد افتراضي 100 طلب لكل 15 دقيقة لكل.

8. storage.config.js — باستخدام (S3-compatible) S3 إعدادات الاتصال بتخزين متوافق مع — @aws-sdk/client-s3 (STORAGE_*) الاتصال مبني بحيث لو متغيرات البيئة الخاصة بالتخزين من غير ما يوقف تشغيل (not configured) "مش موجودة، النظام يعتبر التخزين "غير مُفعّل Phase والاستخدام الفعلي يبدأ في provisioning الحقيقية لسه معملتش S3 السيرفر، لأن بيانات 2.

9. scheduler.core.js — محرك جدول مركزي واحد (registerJob, startJob, stopJob, startAll, stopAll, listJobs) على node-cron مبني على (Phase 4) — فعلية فيه jobs هما اللي هيسجلوا (تجديد الدفعات) Phase 5 و (انتهاء صلاحية الطلبات) Phase 4.

10. env.config.js — يتأكد من وجود المتغيرات المطلوبة dotenv عن طريق env. بيحمّل (fail fast) عند بداية تشغيل السيرفر، ولو ناقصة السيرفر بيوقف فوراً (حالياً MONGODB_URI) (STORAGE_*) بدل ما يشتغل بإعدادات ناقصة ويطلع أخطاء غامضة بعدين. متغيرات التخزين. بتتفحص كمجموعة واحدة لكن مش إجبارية لتشغيل السيرفر.

11. database.config.js — مع إعادة محاولة تلقائية، MongoClient عبر MongoDB الاتصال بـ (event listeners) لحد 5 مرات بفواصل 5 ثواني بين كل محاولة، ومستمعين (retry) (error, disconnected, reconnected)، وداالة isConnected() حالة الاتصال الحالية.

12. app.entry.js — Express، بيبي تطبيق — helmet (headers حماية ال)، cors، body parsers، request logger، rate limiter، الصحة، نقطة فحص /health، ونقطة دي بترجع حالة السيرفر + حالة اتصال قاعدة البيانات + حالة التخزين. الملف مجهز عشان الموديولات error handler بتاعتها فيه بسهولة، وفي الآخر بيركب ال routers تركب ال (auth, students...) الجاية.

13. server.entry.js — بيتصل بقاعدة البيانات (process entrypoint): نقطة تشغيل العملية — scheduler الأول، بعدين يشغل ال وبعدين يفتح السيرفر على البورت المحدد، مع معالجة إيقاف graceful shutdown) عند استقبال SIGINT/SIGTERM.

أي قرارات تقنية اتأخذت أثناء التنفيذ

- المواصفة الأصلية سمحت بالاختيارين Redis+Bull بدل node-cron: **محرك الجدولة** package.json، لأنه أصلاً موجود في node-cron اخترنا ("Redis + Bull، أو node-cron") في مرحلة التأسيس. لو احتجنا لاحقاً (Redis تشغيل) وملوش احتياج لبنية تحتية إضافية هو المكان الوحيد اللي هيتغير scheduler.core.js، concurrency، وretries حقيقي بـ queue فيه المحرك من غير ما نلمس أي كود بينادي عليه.
- **file- بدل الدسك في (memoryStorage) تخزين الملفات في الذاكرة** من غير ما تتخزن مؤقتاً على S3 عشان الملفات تتبع مباشرة لـ upload.util.js، السيرفر.
- **غير إجبارية لتشغيل السيرفر.** رغم إن المواصفة قالت (S3) متغيرات التخزين

لأن بيانات (fail fast) دلوقتي"، معملناش الاتصال ده إجباري storage bucket "اتصل بال Phase 2. الحقيقية لسه مش متوفرة من العميل، والاستخدام الفعلي بيبدأ S3

لو المتغيرات مش موجودة، (graceful) بيتصرف بشكل متسامح storage.config.js health check. من غير ما يمنع تشغيل السيرفر أو الـ { configured: false } بيرجع

- جديدة لأنها dependency كـ package.json **أضافت لـ** aws-sdk/client-s3 @aws مكنبة بيئة التنفيذ الحالية — node_modules مش كانت موجودة أصلاً، لكن **متثبتش فعلياً في** على أي طلب Forbidden بيرجع خطأ 403 npm registry معندهاش وصول لـ (sandbox) على جهاز فيه إنترنت عادي قبل تشغيل السيرفر لأول npm install لازم تشغيل. (تثبت مرة.
- **المواصفة الأصلية كتبت المسار middlewares. هيكل مجلد الـ** src/shared/middlewares/، لكن المشروع كان جاهز بمجلد src/middleware/ (بدون من الهيكل الأولية اللي كانت موجودة قبل المرحلة دي. اتقرر نكمل على الهيكل (shared) الموجود فعلياً في المشروع بدل ما نعمل نقل/إعادة تسمية مش مطلوبة صراحة، وده "موثّق تحت كـ" انحراف عن الخطة
- **الملف ده كان موجود في المشروع .gitignore. لـ** atlas-credentials.env **إضافة** الحقيقية بالنص الصريح. أضاف لـ Atlas وفيه بيانات اتصال (git غير متتبّع في) كإجراء احترازي إضافي عشان يتضمن عدم رفعه بالغلط لاحقاً .gitignore.

الاختبارات اللي اتعملت والنتائج

الاختبار

تحميل كل ملفات المرحلة عن طريق
(فحص الأخطاء النحوية والاعتمادية) require()
مباشرة وفحص نقطة app.entry.js تشغيل
/health

response.util و error-handler فحص أن
يبرجعوا القيم المتوقعة scheduler.core
MongoDB Atlas محاولة اتصال فعلي بـ
المُعطى MONGODB_URI بالـ

بعد كل التعديلات git status فحص

النتيجة

نجح — كل الملفات اتحمّلت من غير أخطاء
نجح — رجعت الشكل الصحيح:
{ "success": true, "message": "Degraded",
 "data": { "server": "up", "database":
 { "connected": false }, "storage":
 { "configured": false, ... } } } مع HTTP 503
لأن قاعدة البيانات مش متصلة في بيئة (الاختبار)
نجح

بيئة التنفيذ (لم يكتمل — قيد فحصي)
معندهاش وصول شبكة (sandbox الحالية
TCP أو DNS لـ raw network egress) خام
خارج نطاق محدود جدّاً من الدومينات
المسموح بيها. محاولة الاتصال فشلت
بسبب querySrv ECONNREFUSED فوراً بخطأ
لطلب البحث عن sandbox رفض شبكة الـ
MongoDB Atlas الخاص بـ DNS SRV سجل
مش بسبب خطأ في بيانات الاتصال أو
في الكود.

atlas-credentials.env و .env — نظيف
والملفات المعدّلة فقط، git مستثنين صح من
Phase 0 اللي في نطاق

أي انحراف عن الخطة الأصلية

1. **middlewares** (src/middleware/ بدل src/shared/middlewares/) — تم الإبقاء عليه لأنه كان موجود مسبقًا في هيكلية المشروع، بدل عمل تغيير هيكلي غير مطلوب صراحة في التعليمات.
2. **MongoDB Atlas من الاتصال الحي بـ (end-to-end) عدم التحقق الفعلي** ده
شغال ومتصل فعليًا health check الانحراف الأهم في المرحلة دي: طلب مني تأكيد إن الـ
اللي يشتغل فيها محجوب عنها الوصول لأي (sandbox) بقاعدة البيانات، لكن بيئة التنفيذ
خارج نطاق ضيق جدًا من الدومينات، وده مش له علاقة (TCP أو DNS) اتصال شبكة خام
:بيانات الاتصال أو بالكود نفسه. تم التحقق من
الصح بالضبط بالشكل القياسي لـ MONGODB_URI أن الكود يحاول الاتصال بالـ
Mongoose.
سليم لما database.isConnected() بترجع الشكل الصحيح ومنطق /health أن
جربناه بمحاكاة عدم الاتصال. لكن لسه محتاج تأكيد فعلي من جهاز/بيئة عندها
npm بتشغيل (staging جهاز العميل نفسه، أو سيرفر) وصول إنترنت عادي
والتأكد إن http://localhost:5000/health ثم فتح npm start && install
database.connected بترجع true.
3. **node_modules مضافة لكن غير مثبتة فعليًا في @aws-sdk/client-s3 مكتبة**
بسبب node_modules مضافة لكن غير مثبتة فعليًا في @aws-sdk/client-s3 مكتبة
في بيئة فيها إنترنت npm install نفس قيد الشبكة، لحد ما يتعمل.

الحالة النهائية للمرحلة

(Fully complete) مكتملة بالكامل.

اتبينت واتفحصت محليًا بنجاح من غير أخطاء. الجزء اللي كان محتاج (app.entry, server.entry, config, shared, middleware, app.entry, server.entry) الكود كامل 100% ومطابق للمواصفة، وكل الوحدات
تم التحقق منه فعليًا من — MongoDB Atlas تأكيد إضافي — الاتصال الفعلي الحي بقاعدة بيانات
بتاريخ 30-07-2026، ورجع (sandbox بيئة فيها إنترنت عادي، خارج الـ) جهاز العميل
التفاصيل الكاملة موجودة في قسم "التحقق" HTTP 200 مع database.connected: true
النهائي من الاتصال الحي" في آخر التقرير. المرحلة صفر تُعتبر مقفولة بالكامل.

التحقق النهائي من الاتصال الحي

بيئة فيها) هذا القسم اتضاف بعد التحقق الفعلي، وبيوتق نتيجة تشغيل السيرفر على جهاز العميل
(المستخدم أثناء التطوير sandbox إنترنت عادي، مش الـ).
timestamp حسب الـ) UTC تاريخ ووقت التجربة: 30-07-2026، الساعة 21:06:23 بتوقيت
(نفسها /health في استجابة).

الخطوات اللي اتنفذت على جهاز العميل:

1. من غير أخطاء تثبيت. @aws-sdk/client-s3 بما فيها) نجح، اتضافت 25 حزمة — npm i وهي حزم (30 vulnerabilities: 9 moderate, 21 high) عادية npm audit ظهرت تحذيرات Phase 0 لا تمنع التشغيل ومش جزء من نطاق (transitive dependencies) فرعية معروفة. تُترك للمراجعة في مرحلة لاحقة لو لزم الأمر —
2. npm start — السيرفر اشتغل بنجاح:

```
[database.config] MongoDB connected (sakanify)
[server.entry] Sakanify backend listening on port 5000 (development)
2026-07-30T21:06:23.524Z GET /health 200 14.8ms
```

الـ Response الكامل من الـ GET /health:

```
{
  "success": true,
  "message": "OK",
  "data": {
    "server": "up",
    "env": "development",
    "database": {
      "connected": true
    },
    "storage": {
      "configured": false,
      "connected": false,
      "message": "Storage env vars not set"
    },
    "timestamp": "2026-07-30T21:06:23.515Z"
  }
}
```

التأكيد:

- database.connected = true (الـ false بدل) (اللي كانت ظاهرة وقت الاختبار من الـ false بدل).
- HTTP status = 200 OK (بدل 503 اللي كانت راجعة وقت عدم اتصال قاعدة البيانات).
- storage.configured = false — لأن متغيرات STORAGE_* متوقع وسليم، (لما يبدأ الاستخدام الفعلي لتخزين الملفات Phase 2 هيتم توفيرها في) متوفرة.

اتأكد بنجاح من بيئة حقيقية فيها إنترنت عادي. MongoDB Atlas **الخلاصة:** الاتصال الحي بـ الانحراف رقم 2 المذكور فوق (عدم التحقق الفعلي من الاتصال الحي) بقي محلول بالكامل، اثبتت بنجاح كمان. **الحالة النهائية للمرحلة** (الانحراف رقم 3) @aws-sdk/client-s3 ومكتبة **صفر: مكتملة بالكامل**.